PYTHON

The Complete Beginner to Advanced Study Guide

Theory - Examples - Multiple-Choice Questions - Practical Tasks

Includes full setup guides for Windows, macOS, and Linux

Table of Contents

How to Use This Guide

This study guide takes you from absolute beginner to advanced, job-ready Python developer. It is organized into five parts:

- Part 1 - Setup: installing Python, VS Code, extensions, and dependencies on Windows, Mac, and Linux.
- Part 2 - Fundamentals: variables, data types, control flow, functions, and core data structures.
- Part 3 - Intermediate: error handling, files, modules, object-oriented programming, and comprehension.
- Part 4 - Advanced: decorators, generators, concurrency, databases, testing, and type hints.
- Part 5 - Practical Projects: build real programs that combine everything you have learned.

Every chapter follows the same structure: a plain-English explanation, a worked example, multiple-choice questions to test recall, and a practical task to build real skill. Answers to all multiple-choice questions are collected in the Answer Key at the end of the guide.

> **Tip:** Don't just read the code examples - type them out yourself in VS Code and run them. Typing builds muscle memory that reading never does.
>
> **Tip:** If a concept doesn't click the first time, move on to the practical task for that chapter and come back to the theory afterwards. Doing often teaches faster than reading.

PART 1: SETTING UP YOUR PYTHON ENVIRONMENT

Before you can write a single line of Python, you need three things installed on your computer: the Python interpreter itself, a code editor (we use Visual Studio Code, the most popular free editor for Python), and a small set of helper tools called dependencies or packages. This part walks through every step for Windows, Mac, and Linux.

1.1 Installing Python

Python is a free, open-source programming language. You install the 'interpreter', which is the program that reads your .py files and runs them.

Installing Python on Windows

1. Open your web browser and go to python.org/downloads.
2. Click the yellow 'Download Python 3.x.x' button - it automatically detects Windows.
3. Run the downloaded installer (python-3.x.x-amd64.exe).
4. On the very first installer screen, TICK the checkbox at the bottom that says 'Add python.exe to PATH'. This is the single most important step - skipping it causes the 'python is not recognized' error later.
5. Click 'Install Now' and wait for the installation to finish.
6. Once done, click 'Close'.
7. Verify the install: open Command Prompt (press Win key, type cmd, press Enter) and type the command below.

```
python --version
```

You should see something like Python 3.12.4 printed back. If you instead see an error, restart your computer (so the PATH change takes effect) and try again.

Installing Python on Mac

macOS ships with an old, outdated version of Python that you should not use for development. Install a fresh copy instead.

8. Go to python.org/downloads and click 'Download Python 3.x.x' - it detects macOS automatically.
9. Open the downloaded .pkg file and click through the installer (Continue -> Continue -> Agree -> Install).
10. Enter your Mac password when prompted, then click 'Close' once finished.
11. Open the Terminal app (press Cmd+Space, type Terminal, press Enter) and verify:

```
python3 --version
```

On Mac, the command is python3, not python, because the system reserves 'python' for its old built-in copy. An alternative, often cleaner approach is to install Python via Homebrew:

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
brew install python
```

Installing Python on Linux

Most Linux distributions come with Python 3 pre-installed. Check first:

```
python3 --version
```

If it is missing or outdated, install it using your distribution's package manager.

Debian / Ubuntu:

```
sudo apt update
sudo apt install python3 python3-pip python3-venv
```

Fedora:

```
sudo dnf install python3 python3-pip
```

Arch Linux:

```
sudo pacman -S python python-pip
```

> **Note:** On Linux and Mac, pip is invoked as pip3 unless you are inside a virtual environment (covered below), where plain pip works fine.

1.2 Installing Visual Studio Code

Visual Studio Code (VS Code) is a free, lightweight code editor made by Microsoft. It is the most widely used editor for Python thanks to its excellent extensions.

Installing VS Code on Windows

12. Go to code.visualstudio.com.
13. Click the blue 'Download for Windows' button.
14. Run the downloaded VSCodeUserSetup-x64-x.x.x.exe installer.
15. Accept the license agreement and click Next through the screens.
16. On the 'Select Additional Tasks' screen, tick 'Add to PATH' and 'Register Code as an editor for supported file types' (usually checked by default).
17. Click Install, then Finish. VS Code will open automatically.

Installing VS Code on Mac

18. Go to code.visualstudio.com and click 'Download for Mac'.
19. Open the downloaded .zip file - it extracts to Visual Studio Code.app.
20. Drag Visual Studio Code.app into your Applications folder.
21. Open it from Applications (or Spotlight search).
22. Optional but recommended: open VS Code, press Cmd+Shift+P, type 'Shell Command', and select 'Shell Command: Install code command in PATH'. This lets you type 'code .' in Terminal to open any folder in VS Code.

Installing VS Code on Linux

Debian / Ubuntu (.deb):

```
sudo apt update
sudo apt install software-properties-common apt-transport-https wget -y
wget -qO- https://packages.microsoft.com/keys/microsoft.asc | gpg --dearmor >
packages.microsoft.gpg
sudo install -D -o root -g root -m 644 packages.microsoft.gpg
/etc/apt/keyrings/packages.microsoft.gpg
sudo sh -c 'echo "deb [arch=amd64,arm64,armhf signed-
by=/etc/apt/keyrings/packages.microsoft.gpg] https://packages.microsoft.com/repos/code
stable main" > /etc/apt/sources.list.d/vscode.list'
```

```
sudo apt update
sudo apt install code
```

Fedora / RHEL (.rpm):

```
sudo rpm --import https://packages.microsoft.com/keys/microsoft.asc
sudo sh -c 'echo -e "[code]\nname=Visual Studio
Code\nbaseurl=https://packages.microsoft.com/yumrepos/vscode\nenabled=1\ngpgcheck=1\ngpgkey=
https://packages.microsoft.com/keys/microsoft.asc" > /etc/yum.repos.d/vscode.repo'
sudo dnf check-update
sudo dnf install code
```

Alternatively, on any distro that supports it, the simplest method is the Snap Store or Flathub:

```
sudo snap install --classic code
```

1.3 Essential VS Code Extensions for Python

Open the Extensions panel in VS Code with Ctrl+Shift+X (Windows/Linux) or Cmd+Shift+X (Mac), search for each name below, and click Install.

Extension	Publisher	Why you need it
Python	Microsoft	Core extension: syntax highlighting, IntelliSense, debugging, linting, run button.
Pylance	Microsoft	Fast, accurate autocompletion and type-checking (installs automatically with Python extension).
Jupyter	Microsoft	Run notebook (.ipynb) cells directly inside VS Code - great for data science and experimentation.
Python Debugger	Microsoft	Dedicated debugging engine, works alongside the Python extension for breakpoints and step-through.
autoDocstring	Nils Werner	Auto-generates docstring templates for functions and classes.
indent-rainbow	oderwat	Colour-codes indentation levels so Python's whitespace rules are easy to see at a glance.
GitLens	GitKraken	Rich Git history, blame annotations, and version control insight inside the editor.
Code Runner	Jun Han	Adds a one-click 'Run' button for quick script execution without a full debug session.
Black Formatter	Microsoft	Auto-formats your code to a consistent, community-standard style on save.
Ruff	Astral Software	Extremely fast linter that flags errors and style issues as you type.

Recommended minimum: If you only install three, install Python, Pylance, and Black Formatter. Add the rest as you grow.

1.4 Dependencies, pip, and Virtual Environments

A 'dependency' is a piece of code someone else wrote and packaged, that your project relies on (for example, the 'requests' library for making web requests). Python's package manager, pip, installs these for you automatically.

Checking pip is installed

```
python -m pip --version
```

pip ships with Python automatically from Python 3.4 onward, so this should already work. If it doesn't, reinstall Python and make sure 'pip' is ticked during setup (Windows) or run 'python3 -m ensurepip --upgrade' (Mac/Linux).

Installing a package

```
pip install requests
```

Replace 'requests' with the name of any package you need. Popular ones you will meet in this guide include:

- requests - for making HTTP calls to web APIs
- numpy - for fast numerical arrays and maths
- pandas - for working with tabular data (spreadsheets/CSVs)
- matplotlib - for charts and graphs
- pytest - for writing automated tests
- flask / django - for building web applications
- black and ruff - for formatting and linting (used from the command line as well as VS Code)

Virtual environments - why and how

A virtual environment is an isolated, self-contained copy of Python for a single project. It stops packages from one project clashing with packages from another (for example, Project A needing Django 3 while Project B needs Django 5). You should create one for every serious project.

Create and activate a virtual environment:

Windows (Command Prompt):

```
python -m venv venv
venv\Scripts\activate
```

Mac / Linux (Terminal):

```
python3 -m venv venv
source venv/bin/activate
```

Once activated, you will see (venv) at the start of your terminal prompt. Any 'pip install' you run now installs only inside this project's folder. To leave the environment, type 'deactivate'.

Saving and restoring dependencies with requirements.txt

So that anyone else (or future-you) can recreate your exact setup, freeze your installed packages into a text file:

```
pip freeze > requirements.txt
```

And to install everything listed in that file on a new machine:

```
pip install -r requirements.txt
```

Selecting your interpreter in VS Code

23. Open your project folder in VS Code (File > Open Folder).
24. Press Ctrl+Shift+P (Cmd+Shift+P on Mac) to open the Command Palette.
25. Type 'Python: Select Interpreter' and press Enter.
26. Choose the interpreter inside your venv folder (it will be listed as 'venv: venv') so VS Code and your terminal always agree on which Python is running.

Writing and running your first file

27. In VS Code, create a new file and save it as hello.py.
28. Type the following code:

```
print("Hello, World!")
```

29. Run it: click the Run (triangle) button in the top-right corner, or right-click the editor and choose 'Run Python File in Terminal', or simply type in the integrated terminal:

```
python hello.py
```

If 'Hello, World!' prints in the terminal below, your entire environment is correctly set up. Congratulations - you are ready to start Part 2.

1.5 Debugging Python in VS Code

A debugger lets you pause your program mid-execution, inspect variable values, and step through code line by line - far faster than sprinkling print() statements everywhere.

30. Open the file you want to debug in VS Code.
31. Click in the space just to the left of a line number to set a breakpoint - a red dot appears.
32. Open the Run and Debug panel (Ctrl+Shift+D / Cmd+Shift+D) and click 'Run and Debug', then choose 'Python File'.
33. Execution will pause at your breakpoint. Use the floating toolbar to Step Over (F10), Step Into (F11), or Continue (F5).
34. While paused, hover over any variable to see its current value, or check the 'Variables' panel on the left.
35. Add expressions to the 'Watch' panel to track specific values as you step through the code.

> **Tip:** Use 'Step Into' (F11) when you want to follow execution inside a function call; use 'Step Over' (F10) when you trust that function works and just want to move past it.

1.6 Version Control Basics: Git and GitHub

Git tracks changes to your code over time, and GitHub hosts your code online so you can back it up, collaborate, and showcase your work. Every professional developer uses both daily.

Installing Git

- Windows: download and run the installer from git-scm.com (accept the default options).
- Mac: run 'git --version' in Terminal - macOS will offer to install the Xcode Command Line Tools, which include Git. Alternatively use Homebrew: brew install git.
- Linux: sudo apt install git (Debian/Ubuntu) or sudo dnf install git (Fedora).

Core Git workflow

```
git init                          # start tracking a project
git add .                         # stage all changed files
git commit -m "Initial commit"    # save a snapshot with a message
git branch feature-login          # create a new branch
git checkout feature-login        # switch to that branch
git push origin main              # upload commits to GitHub
git pull                          # download the latest changes
git clone https://github.com/user/repo.git   # copy a repo to your machine
```

With the GitLens extension installed (see 1.3), VS Code shows inline blame annotations, commit history, and a visual Source Control panel (Ctrl+Shift+G) so you rarely need the raw commands above once you're comfortable with the basics.

Q1. What is the difference between Git and GitHub?

A) They are the same thing

B) Git is the version-control tool that runs locally; GitHub is a website that hosts Git repositories online

C) GitHub is only for Python

D) Git only works on Linux

Q2. Which command saves a snapshot of your staged changes with a message?

A) git push

B) git add

C) git commit -m "message"

D) git init

Practical Task 1: Debugging and Git Practice

Practise the two most important daily developer workflows.

- Write a small script with a deliberate logic error (e.g. an off-by-one loop bug).
- Set a breakpoint before the bug and use the VS Code debugger to step through and identify the problem.
- Fix the bug, then initialise a Git repository in the project folder.
- Make an initial commit, then create a GitHub account (if you don't have one) and push the project to a new repository.

1.7 Troubleshooting Common Setup Problems

Problem	Likely cause	Fix
'python' is not recognized as a command	Python wasn't added to PATH during install	Reinstall Python and tick 'Add python.exe to PATH', or add it manually via System Environment Variables
'pip' is not recognized	pip isn't on PATH, or Python was installed without it	Run 'python -m ensurepip --upgrade', or reinstall with pip included
VS Code runs the wrong Python version	Wrong interpreter selected	Ctrl+Shift+P -> 'Python: Select Interpreter' -> choose the correct one
ModuleNotFoundError after pip install	Package installed outside the active virtual environment	Activate your venv first, then reinstall the package inside it
Permission denied errors on Mac/Linux	Trying to install system-wide without sufficient rights	Use a virtual environment instead of installing globally, avoiding the need for sudo pip install
Terminal shows (venv) but VS Code still uses global Python	VS Code interpreter setting out of sync with the terminal	Reload VS Code (Ctrl+Shift+P -> 'Reload Window') after selecting the venv interpreter

> **General rule:** The vast majority of beginner setup issues come down to one of two things: Python not being on PATH, or a virtual environment not being activated. Check those two first.

Chapter 1 Quiz: Setup

Q3. On Windows, which installer checkbox must you tick to avoid the 'python is not recognized' error?

A) Install pip
B) Add python.exe to PATH
C) Install for all users
D) Create desktop shortcut

Q4. On macOS, why is the command python3 used instead of python?

A) python3 is faster
B) macOS reserves 'python' for its outdated system copy
C) python only works on Windows
D) There is no difference

Q5. What is the main purpose of a virtual environment?

A) To make Python run faster
B) To isolate a project's packages from other projects
C) To connect to the internet
D) To compile Python into an executable

Q6. Which command installs every package listed inside requirements.txt?

A) pip freeze requirements.txt
B) pip install -r requirements.txt
C) python requirements.txt
D) pip get requirements.txt

Practical Task 2: Environment Setup

Set up your complete Python development environment from scratch.

- Install Python and confirm the version with python --version (or python3 --version).
- Install VS Code and the Python + Pylance extensions.
- Create a project folder called python_guide, open it in VS Code, and create a virtual environment inside it.
- Activate the virtual environment and install the requests package.
- Freeze your dependencies into requirements.txt.
- Write and run a hello.py file that prints your name and today's date.

PART 2: PYTHON FUNDAMENTALS

This part covers everything a beginner needs to write real, working Python programs: storing data, making decisions, repeating actions, organising code into functions, and using Python's core data structures.

2.1 Variables and Data Types

A variable is a named container that stores a value in memory. Python is 'dynamically typed', meaning you don't have to declare a type upfront - Python figures it out from the value you assign.

```
name = "Byron"        # str (text)
age = 21               # int (whole number)
height = 1.78          # float (decimal number)
is_student = True      # bool (True/False)
nothing = None         # NoneType (absence of a value)

print(type(name))      # <class 'str'>
```

Core built-in data types

Type	Example	Description
int	42	Whole numbers, positive or negative
float	3.14	Numbers with a decimal point
str	"hello"	Text, wrapped in quotes
bool	True / False	Logical true/false value
list	[1, 2, 3]	Ordered, changeable collection
tuple	(1, 2, 3)	Ordered, unchangeable collection
dict	{"a": 1}	Key-value pairs
set	{1, 2, 3}	Unordered collection of unique items
NoneType	None	Represents 'no value'

Naming rules: variable names must start with a letter or underscore, can contain letters/numbers/underscores, are case-sensitive, and cannot be a reserved keyword (like if, for, class). Python convention (PEP 8) uses snake_case for variable names, e.g. total_score.

Q7. What data type does Python assign to is_active = True?

- **A)** str
- **B)** int
- **C)** bool
- **D)** float

Q8. Which of these is a valid Python variable name?

- **A)** 2total
- **B)** total-score
- **C)** _total
- **D)** class

Practical Task 3: Personal Profile Variables

Create variables describing yourself and print them in a formatted sentence using an f-string.

- Create variables for name, age, city, and favourite_language.
- Print: 'My name is X, I am Y years old, I live in Z, and I love W.' using an f-string.
- Use type() to print the data type of each variable.

2.2 Operators

Arithmetic operators

Operator	Meaning	Example
+	Addition	5 + 2 -> 7
-	Subtraction	5 - 2 -> 3
*	Multiplication	5 * 2 -> 10
/	Division (always returns float)	5 / 2 -> 2.5
//	Floor division	5 // 2 -> 2
%	Modulus (remainder)	5 % 2 -> 1
**	Exponent	5 ** 2 -> 25

Comparison operators

Operator	Meaning
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal

Logical operators

```
age = 20
has_id = True

if age >= 18 and has_id:
    print("Entry allowed")

if age < 13 or not has_id:
    print("Entry denied")
```

Q9. What does 17 // 4 evaluate to?

A) 4.25

B) 4

C) 1

D) 5

Q10. What does 17 % 4 evaluate to?

A) 4

B) 0

C) 1

D) 4.25

2.3 Strings

Strings are sequences of characters. Python provides powerful built-in methods for manipulating text.

```
greeting = "Hello, World!"
print(greeting.lower())      # hello, world!
print(greeting.upper())      # HELLO, WORLD!
print(greeting.replace("World", "Python"))  # Hello, Python!
print(greeting.split(","))   # ['Hello', ' World!']
print(len(greeting))         # 13
print(greeting[0])           # H  (indexing)
print(greeting[7:12])        # World (slicing)
print(greeting[::-1])        # !dlroW ,olleH  (reversed)

name = "Byron"
age = 21
print(f"{name} is {age} years old")   # f-strings: the modern way to format text
```

Common string methods

Method	What it does
.strip()	Removes leading/trailing whitespace
.join()	Joins a list of strings with a separator
.find()	Returns the index of a substring, or -1
.startswith() / .endswith()	Checks the start/end of a string
.isdigit() / .isalpha()	Checks if a string is all digits/letters
.format()	Older-style string formatting

Q11. What does "Hello"[1:4] return?

A) Hell

B) ello

C) ell

D) Hello

Practical Task 4: String Manipulator

Write a program that takes a sentence and reports on it.

- Ask the user to input a sentence using input().
- Print the sentence in all uppercase and all lowercase.
- Print the total number of characters and the number of words (hint: use .split()).
- Print the sentence reversed.

2.4 Input and Output

```
name = input("What is your name? ")
print("Hello, " + name + "!")

# input() always returns a string, so convert numbers explicitly:
age_text = input("How old are you? ")
age = int(age_text)
print(f"Next year you will be {age + 1}")
```

Common beginner error: Forgetting to convert input() to int or float before doing maths on it causes a TypeError, because input always returns a string.

2.5 Control Flow: if, elif, else

```
score = 72

if score >= 90:
    print("Grade: A")
elif score >= 70:
    print("Grade: B")
elif score >= 50:
    print("Grade: C")
else:
    print("Grade: F")
```

Python uses indentation (4 spaces by convention) instead of curly braces to define code blocks. Consistent indentation is not optional - it is part of the syntax.

Q12. In Python, what defines which lines belong inside an if block?

A) Curly braces {}

B) Indentation

C) Semicolons

D) Parentheses

Practical Task 5: Grade Calculator

Build a small program that classifies a score into a letter grade.

- Ask the user to input a score between 0 and 100.
- Use if/elif/else to print the correct letter grade (A/B/C/D/F).
- Add validation: if the number is outside 0-100, print an error message instead.

2.6 Loops: for and while

for loops

```
for i in range(5):
    print(i)          # prints 0 1 2 3 4

fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)

for index, fruit in enumerate(fruits):
    print(index, fruit)   # 0 apple / 1 banana / 2 cherry
```

while loops

```
count = 0
while count < 5:
    print(count)
    count += 1

# break and continue
number = 0
while True:
    number += 1
    if number == 3:
        continue    # skip printing 3
    if number > 5:
        break       # stop the loop entirely
    print(number)
```

Q13. What does range(5) produce when looped over?

A) 1,2,3,4,5

B) 0,1,2,3,4

C) 0,1,2,3,4,5

D) 5,4,3,2,1,0

Q14. Inside a loop, what does the 'continue' keyword do?

A) Stops the loop completely

B) Skips the rest of the current iteration and moves to the next one

C) Pauses the program

D) Restarts the loop from zero

Practical Task 6: Number Games

Practise loops with two small challenges.

- Print all even numbers from 1 to 50 using a for loop and the modulus operator.
- Write a while loop that keeps asking the user for a password until they type 'exit123'.
- Write a for loop using enumerate() that numbers a shopping list starting from 1, not 0.

2.7 Functions

A function is a reusable, named block of code. Functions let you avoid repeating yourself and make programs easier to read and test.

```
def greet(name, greeting="Hello"):
    """Return a greeting message for the given name."""
    return f"{greeting}, {name}!"

print(greet("Byron"))                 # Hello, Byron!
print(greet("Byron", "Hi"))           # Hi, Byron!
print(greet(name="Byron", greeting="Yo"))  # keyword arguments

# *args and **kwargs for flexible argument counts
def add_all(*numbers):
    return sum(numbers)

print(add_all(1, 2, 3, 4))   # 10

def describe(**details):
    for key, value in details.items():
        print(f"{key}: {value}")

describe(name="Byron", role="Developer")
```

Scope

Variables created inside a function are 'local' - they only exist while the function runs and cannot be accessed outside it. Variables created outside any function are 'global'.

Q15. What keyword sends a value back from a function to the code that called it?

A) print

B) return

C) yield only

D) output

Q16. What does *args allow a function to accept?

A) Exactly one argument

B) Any number of positional arguments

C) Only keyword arguments

D) No arguments at all

Practical Task 7: Function Toolkit

Write a small library of reusable functions.

- Write is_even(number) that returns True or False.
- Write calculate_total(*prices) that returns the sum of any number of prices passed in.
- Write build_profile(name, **extra_info) that returns a dictionary combining the name with any extra keyword info.
- Call each function and print the results.

2.8 Lists, Tuples, Sets, and Dictionaries

Lists - ordered and changeable

```
numbers = [3, 1, 4, 1, 5]
numbers.append(9)          # add to the end
numbers.insert(0, 100)     # insert at index 0
numbers.remove(1)          # removes the first '1' found
numbers.sort()             # sorts in place
print(numbers)
print(numbers[-1])         # last item
print(len(numbers))        # number of items
squared = [n ** 2 for n in numbers]   # list comprehension
print(squared)
```

Tuples - ordered and unchangeable

```
point = (10, 20)
x, y = point               # unpacking
print(x, y)
# point[0] = 5             # this would raise a TypeError - tuples are immutable
```

Sets - unordered, unique items

```
a = {1, 2, 3}
b = {2, 3, 4}
print(a | b)    # union -> {1, 2, 3, 4}
print(a & b)    # intersection -> {2, 3}
print(a - b)    # difference -> {1}

colours = ["red", "blue", "red", "green"]
unique_colours = set(colours)   # removes duplicates automatically
print(unique_colours)
```

Dictionaries - key/value pairs

```
student = {"name": "Byron", "age": 21, "course": "WIL"}
print(student["name"])
student["age"] = 22             # update a value
student["grade"] = "A"          # add a new key
```

```
for key, value in student.items():
    print(f"{key}: {value}")

print(student.get("email", "Not provided"))  # safe lookup with a default
```

Choosing the right structure

Structure	Ordered?	Changeable?	Duplicates?	Use when...
list	Yes	Yes	Yes	You need a sequence you'll modify
tuple	Yes	No	Yes	You need fixed, protected data (e.g. coordinates)
set	No	Yes	No	You need uniqueness or fast membership tests
dict	Yes (3.7+)	Yes	Keys: No	You need to look values up by name/key

Q17. Which data structure would you choose to store a phone book of name-to-number pairs?

A) list
B) tuple
C) dict
D) set

Q18. Why would removing duplicates from a list often involve converting it to a set?

A) Sets are always faster to loop over
B) Sets cannot contain duplicate values by definition
C) Sets are ordered
D) Lists cannot contain numbers

Q19. What is the key difference between a list and a tuple?

A) Tuples can only store numbers
B) Lists are unchangeable, tuples are changeable
C) Tuples are immutable (cannot be changed after creation), lists are mutable
D) There is no difference

Practical Task 8: Student Records System

Combine all four data structures in one mini-program.

- Create a list of dictionaries, each representing a student with name, age, and grades (a list of numbers).
- Loop through the list and print each student's name with their average grade.
- Use a set to find and print all the unique grades earned across every student.
- Store the (min_grade, max_grade) as a tuple for each student and print it.

2.9 Type Conversion (Casting)

Type conversion changes a value from one data type to another. This is essential when combining text and numbers, or processing input().

```
age_text = "21"
age_number = int(age_text)          # str -> int
price = float("19.99")              # str -> float
count_text = str(42)                # int -> str
is_valid = bool(1)                  # 1 -> True, 0 -> False

# Common gotcha:
# print("Age: " + 21)   # TypeError: cannot concatenate str and int
print("Age: " + str(21))              # Works: Age: 21
print(f"Age: {21}")                   # Better: f-strings convert automatically
```

Q20. What will int("3.5") do in Python?

A) Return 3

B) Return 4 (rounds up)

C) Raise a ValueError

D) Return 3.5

2.10 Writing Clean, Readable Code

Code is read far more often than it is written. Following Python's official style guide, PEP 8, makes your code easier for others (and future-you) to understand.

- Use descriptive names: total_price instead of tp.
- Keep functions short and focused on a single task.
- Add comments to explain 'why', not 'what' (the code already shows what it does).
- Use docstrings (triple-quoted strings right after a def or class) to document purpose, arguments, and return values.

```
def calculate_discount(price, percent):
    """
    Calculate the discounted price.

    Args:
        price (float): The original price.
        percent (float): Discount percentage (0-100).

    Returns:
        float: The price after applying the discount.
    """
    return price - (price * percent / 100)
```

2.11 Nested Data Structures

Real-world data is rarely flat. Lists of dictionaries, dictionaries of lists, and dictionaries of dictionaries let you model complex information - this is also exactly the shape of JSON data used by most web APIs.

```
company = {
    "name": "Acme Corp",
    "employees": [
        {"name": "Byron", "role": "Developer", "skills": ["Python", "SQL"]},
        {"name": "Alex", "role": "Designer", "skills": ["Figma", "CSS"]},
    ],
}
```

```
# Accessing nested values
print(company["employees"][0]["name"])          # Byron
print(company["employees"][0]["skills"][1])       # SQL

# Looping through nested structures
for employee in company["employees"]:
    skills_text = ", ".join(employee["skills"])
    print(f"{employee['name']} ({employee['role']}): {skills_text}")
```

Q21. Given company["employees"][0]["skills"][1] in the example above, what does it return?

A) Byron

B) Developer

C) Python

D) SQL

Practical Task 9: Nested Data Practice

Model and query a small nested dataset.

- Build a dictionary representing a school with a list of classes, each containing a list of student names.
- Write a loop that prints every student, grouped under their class name.
- Write a function that counts the total number of students across all classes.

PART 3: INTERMEDIATE PYTHON

With the fundamentals in place, this part introduces the tools professional developers use every day: handling errors gracefully, reading and writing files, organising code across files, and Object-Oriented Programming (OOP).

3.1 Error Handling: try, except, finally

Errors ('exceptions') happen - bad user input, missing files, network failures. Instead of letting your program crash, you can catch and handle these situations.

```
try:
    number = int(input("Enter a number: "))
    result = 100 / number
    print(result)
except ValueError:
    print("That was not a valid number.")
except ZeroDivisionError:
    print("You cannot divide by zero.")
else:
    print("No errors occurred!")
finally:
    print("This always runs, error or not.")
```

Common built-in exceptions

Exception	When it happens
ValueError	A function receives an argument of the right type but wrong value (e.g. int('abc'))
TypeError	An operation is applied to an incompatible type (e.g. '2' + 2)
ZeroDivisionError	Dividing a number by zero
IndexError	Accessing a list index that doesn't exist
KeyError	Accessing a dictionary key that doesn't exist
FileNotFoundError	Trying to open a file that doesn't exist

Raising your own exceptions

```
def withdraw(balance, amount):
    if amount > balance:
        raise ValueError("Insufficient funds")
    return balance - amount

try:
    withdraw(100, 150)
except ValueError as e:
    print(f"Transaction failed: {e}")
```

Q22. Which block of code always runs, whether or not an exception occurred?

A) try

B) except

C) else

D) finally

Q23. Which exception is raised when you try to access a dictionary key that does not exist?

A) IndexError
B) KeyError
C) ValueError
D) AttributeError

Practical Task 10: Safe Calculator

Build a calculator that never crashes, no matter what the user types.

- Ask the user for two numbers and an operator (+, -, *, /).
- Use try/except to catch invalid numeric input (ValueError).
- Catch division by zero separately with a friendly message.
- Use a while loop so the calculator keeps running until the user types 'quit'.

3.2 Working with Files

```
# Writing to a file
with open("notes.txt", "w") as file:
    file.write("Hello, this is line one.\n")
    file.write("This is line two.\n")

# Reading a file
with open("notes.txt", "r") as file:
    content = file.read()
    print(content)

# Reading line by line
with open("notes.txt", "r") as file:
    for line in file:
        print(line.strip())

# Appending to a file
with open("notes.txt", "a") as file:
    file.write("This line was appended.\n")
```

Why use 'with'? The 'with' statement (a context manager) automatically closes the file for you, even if an error occurs inside the block. Always prefer it over manually calling open()/close().

Working with CSV files

```
import csv

with open("students.csv", "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerow(["Name", "Grade"])
    writer.writerow(["Byron", "A"])

with open("students.csv", "r") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```

Working with JSON

```
import json

data = {"name": "Byron", "skills": ["Python", "JavaScript"]}
```

```
with open("data.json", "w") as file:
    json.dump(data, file, indent=4)

with open("data.json", "r") as file:
    loaded = json.load(file)
    print(loaded["skills"])
```

Q24. Which file mode appends new content to the end of an existing file without erasing it?

A) "r"

B) "w"

C) "a"

D) "x"

Practical Task 11: Contact Book File Storage

Persist data to disk using JSON.

- Write a program that stores contacts (name, phone, email) in a Python dictionary.
- Save the dictionary to a file called contacts.json using the json module.
- Write a second function that loads contacts.json back into a dictionary and prints every contact.
- Add a feature to append a new contact and re-save the file.

3.3 Modules and Packages

A module is simply a .py file containing functions/classes you can reuse. A package is a folder of related modules.

```
# math_utils.py
def add(a, b):
    return a + b

def square(n):
    return n * n

# main.py
import math_utils
print(math_utils.add(2, 3))

from math_utils import square
print(square(4))

# Using the standard library
import math
import random
import datetime

print(math.sqrt(16))
print(random.randint(1, 10))
print(datetime.datetime.now())
```

Useful standard library modules

Module	Purpose
math	Mathematical functions (sqrt, floor, pi, etc.)
random	Generating random numbers and choices

datetime	Working with dates and times
os	Interacting with the operating system and file paths
sys	Interacting with the Python interpreter (arguments, exit codes)
collections	Specialised containers like Counter and defaultdict

Q25. What is the difference between 'import math' and 'from math import sqrt'?

A) No difference at all

B) The first requires math.sqrt(), the second lets you call sqrt() directly

C) The second is faster to run

D) The first only works in Python 2

Practical Task 12: Personal Utility Module

Build and reuse your own module.

- Create a file called string_utils.py with functions: is_palindrome(text) and count_vowels(text).
- In a separate main.py, import your module and call both functions on a few test strings.
- Add a third function of your choice to string_utils.py and use it too.

3.4 Object-Oriented Programming (OOP)

OOP models real-world things as 'objects' that bundle together data (attributes) and behaviour (methods). A 'class' is the blueprint; an 'object' (or instance) is a specific thing built from that blueprint.

```
class Dog:
    species = "Canis familiaris"    # class attribute, shared by all dogs

    def __init__(self, name, age):
        self.name = name             # instance attribute
        self.age = age

    def bark(self):
        return f"{self.name} says Woof!"

    def __str__(self):
        return f"Dog({self.name}, {self.age} years old)"

my_dog = Dog("Rex", 3)
print(my_dog.bark())     # Rex says Woof!
print(my_dog)            # Dog(Rex, 3 years old)
```

The four pillars of OOP

1. Encapsulation - bundling data and methods, hiding internal details

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance     # double underscore = "private" by convention

    def deposit(self, amount):
        self.__balance += amount

    def get_balance(self):
```

```
        return self.__balance

account = BankAccount(100)
account.deposit(50)
print(account.get_balance())   # 150
# account.__balance          # not directly accessible from outside
```

2. Inheritance - a class reusing and extending another class

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return "Some generic sound"

class Cat(Animal):
    def speak(self):                # method overriding
        return f"{self.name} says Meow!"

class Puppy(Animal):
    def __init__(self, name, breed):
        super().__init__(name)      # call the parent constructor
        self.breed = breed

cat = Cat("Whiskers")
print(cat.speak())   # Whiskers says Meow!
```

3. Polymorphism - different classes responding to the same method call in their own way

```
animals = [Cat("Tom"), Animal("Generic")]
for animal in animals:
    print(animal.speak())   # each class provides its own version of speak()
```

4. Abstraction - hiding complex implementation behind a simple interface

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

rect = Rectangle(4, 5)
print(rect.area())   # 20
# Shape() directly would raise TypeError - it is abstract
```

Q26. What does the __init__ method do in a Python class?

A) Deletes an object

B) Runs automatically when a new object is created, to set up its initial attributes

C) Prints the object

D) Is only used for inheritance

Q27. What OOP concept does this describe: 'a Cat class and a Dog class both have a speak() method, but each produces different output'?

A) Encapsulation

B) Polymorphism

C) Abstraction

D) Composition

Q28. What keyword lets a child class call a method from its parent class?

A) parent()

B) super()

C) base()

D) self()

Practical Task 13: Vehicle Class Hierarchy

Practise inheritance and polymorphism with a vehicle system.

- Create a base class Vehicle with attributes make, model, and a method describe().
- Create Car and Motorcycle classes that inherit from Vehicle, each overriding describe() to add a vehicle-specific detail.
- Create a list containing a mix of Car and Motorcycle objects, then loop through and call describe() on each.
- Add a private attribute __mileage to Vehicle with a get_mileage() method to access it (encapsulation).

3.5 Comprehensions and Functional Tools

```
# List comprehension
squares = [x ** 2 for x in range(10)]
evens = [x for x in range(20) if x % 2 == 0]

# Dictionary comprehension
squares_dict = {x: x ** 2 for x in range(5)}

# Set comprehension
unique_lengths = {len(word) for word in ["cat", "dog", "mouse"]}

# lambda - a small, anonymous, one-line function
multiply = lambda a, b: a * b
print(multiply(3, 4))   # 12

# map, filter
numbers = [1, 2, 3, 4, 5]
doubled = list(map(lambda n: n * 2, numbers))
evens_only = list(filter(lambda n: n % 2 == 0, numbers))

# sorted with a key function
people = [("Byron", 21), ("Alex", 19)]
by_age = sorted(people, key=lambda person: person[1])
```

Q29. What does [x ** 2 for x in range(5)] produce?

A) [0, 1, 2, 3, 4]

B) [0, 1, 4, 9, 16]

C) [1, 4, 9, 16, 25]

D) An error

Practical Task 14: Comprehension Practice

Rewrite loops as comprehensions.

- Use a list comprehension to build a list of the cubes of numbers 1 to 10.
- Use a dictionary comprehension to map each word in a list to its length.
- Use filter() and a lambda to extract only the words longer than 4 letters from a list.

3.6 Regular Expressions (regex)

Regular expressions are patterns used to search, match, and manipulate text. Python's built-in 're' module gives you full regex support.

```
import re

text = "Contact us at support@example.com or sales@example.com"

# Find all email addresses
emails = re.findall(r"[\w.-]+@[\w.-]+\.\w+", text)
print(emails)

# Check if a string matches a pattern
phone = "082-555-1234"
if re.match(r"^\d{3}-\d{3}-\d{4}$", phone):
    print("Valid phone number format")

# Replace text using a pattern
censored = re.sub(r"\d", "*", "My PIN is 4821")
print(censored)   # My PIN is ****

# Splitting on multiple delimiters
parts = re.split(r"[,;]\s*", "apples, bananas; cherries")
print(parts)   # ['apples', 'bananas', 'cherries']
```

Common regex symbols

Symbol	Meaning
\d	Any digit (0-9)
\w	Any word character (letters, digits, underscore)
\s	Any whitespace character
.	Any character except newline
*	Zero or more of the previous item
+	One or more of the previous item
^ / $	Start / end of the string
{n}	Exactly n repetitions

Q30. Which re function returns every match of a pattern in a string as a list?

A) re.match()

B) re.search()

C) re.findall()

D) re.compile()

Practical Task 15: Input Validator

Use regex to validate common input formats.

- Write a function is_valid_email(text) using re.match() and an email pattern.
- Write a function is_valid_south_african_phone(text) that checks for a 10-digit number.
- Use re.sub() to strip all punctuation out of a paragraph of text.

3.7 Working with Dates and Times

```
from datetime import datetime, timedelta

now = datetime.now()
print(now)                                # current date and time
print(now.strftime("%Y-%m-%d %H:%M"))   # custom formatted string

birthday = datetime(2005, 6, 15)
age_delta = now - birthday
print(age_delta.days, "days old")

next_week = now + timedelta(days=7)
print(next_week)

parsed = datetime.strptime("2026-09-08", "%Y-%m-%d")
print(parsed.year, parsed.month, parsed.day)
```

Q31. Which datetime method converts a date object into a custom-formatted string?

A) strptime()

B) strftime()

C) now()

D) format()

3.8 Iterables and Iterators

Anything you can loop over with a for loop is 'iterable' (lists, strings, dicts, files). Under the hood, Python calls iter() on it to get an 'iterator', then repeatedly calls next() until it runs out of values.

```
numbers = [10, 20, 30]
iterator = iter(numbers)
print(next(iterator))   # 10
print(next(iterator))   # 20
print(next(iterator))   # 30
# print(next(iterator)) # raises StopIteration - no more items

# Building your own iterable class
class Countdown:
    def __init__(self, start):
        self.current = start

    def __iter__(self):
        return self

    def __next__(self):
        if self.current <= 0:
            raise StopIteration
        self.current -= 1
```

```
        return self.current + 1

for number in Countdown(3):
    print(number)   # 3 2 1
```

Q32. What exception does an iterator raise once it has no more values to give?

A) ValueError

B) IndexError

C) StopIteration

D) EOFError

3.9 Command-Line Arguments and Environment Variables

Scripts often need configuration that shouldn't be hard-coded: file paths, flags, or secrets like API keys. Two common ways to supply this are command-line arguments and environment variables.

```
# script.py
import sys

print("Script name:", sys.argv[0])
print("Arguments:", sys.argv[1:])

# Run with: python script.py hello world
# Output: Arguments: ['hello', 'world']

# A friendlier way using argparse
import argparse

parser = argparse.ArgumentParser(description="Greet a user")
parser.add_argument("name", help="Name of the person to greet")
parser.add_argument("--shout", action="store_true", help="Print in uppercase")
args = parser.parse_args()

greeting = f"Hello, {args.name}!"
print(greeting.upper() if args.shout else greeting)

# Run with: python script.py Byron --shout

import os

api_key = os.environ.get("API_KEY", "default-key-not-set")
print(api_key)

# Setting an environment variable before running (Mac/Linux):
# export API_KEY="abc123"
# Windows (Command Prompt):
# set API_KEY=abc123
```

Security tip: Never hard-code secrets (API keys, passwords) directly in your source code. Use environment variables, or a .env file loaded with the python-dotenv package, and add that file to .gitignore so it never reaches GitHub.

Q33. Why should API keys be stored in environment variables instead of directly in your source code?

A) Environment variables make the code run faster

B) It keeps secrets out of source control, reducing the risk of them leaking publicly

C) Python cannot read strings longer than 20 characters
D) There is no real benefit

PART 4: ADVANCED PYTHON

This part covers the concepts that separate an intermediate coder from a professional Python developer: decorators, generators, concurrency, working with databases and APIs, automated testing, and type hints.

4.1 Decorators

A decorator is a function that wraps another function to add extra behaviour without changing its source code.

```
import time

def timer(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f"{func.__name__} took {end - start:.4f} seconds")
        return result
    return wrapper

@timer
def slow_function():
    time.sleep(1)
    return "Done"

slow_function()   # prints timing info, then runs normally
```

The @timer line is shorthand for slow_function = timer(slow_function). Common real-world decorators include @staticmethod, @classmethod, @property, and Flask's @app.route().

Q34. What does a decorator fundamentally do in Python?

A) Deletes a function

B) Wraps a function to add extra behaviour without modifying its code

C) Converts a function into a class

D) Only works on classes, not functions

Practical Task 16: Logging Decorator

Build a decorator that logs function calls.

- Write a decorator called log_calls that prints the function name and its arguments every time it's called.
- Apply it to two different functions using the @log_calls syntax.
- Extend it to also print the function's return value.

4.2 Generators and Iterators

A generator is a special function that produces a sequence of values lazily (one at a time, on demand) using 'yield' instead of 'return'. This saves memory when working with large or infinite sequences.

```
def count_up_to(limit):
    number = 1
    while number <= limit:
        yield number
        number += 1

for num in count_up_to(5):
```

```
    print(num)     # 1 2 3 4 5, one at a time

# Generator expression (like a list comprehension, but lazy)
squares_gen = (x ** 2 for x in range(1000000))   # uses almost no memory until consumed
print(next(squares_gen))   # 0
print(next(squares_gen))   # 1
```

Q35. What keyword turns a regular function into a generator?

A) return

B) yield

C) generate

D) async

Practical Task 17: Fibonacci Generator

Build a memory-efficient generator.

- Write a generator function fibonacci(limit) that yields Fibonacci numbers up to a given limit.
- Loop over it with a for loop and print every value.
- Compare it in a comment to how you would have written the same thing using a list that stores every value.

4.3 Context Managers

You've already used one context manager: 'with open(...) as file'. You can create your own using a class with __enter__/__exit__, or more simply with the contextlib module.

```
from contextlib import contextmanager

@contextmanager
def timer_context(label):
    import time
    start = time.time()
    yield
    print(f"{label} took {time.time() - start:.4f}s")

with timer_context("My block"):
    total = sum(range(1000000))
```

Q36. What is the main benefit of using a context manager (the 'with' statement)?

A) It makes code run faster

B) It guarantees setup and cleanup code runs, even if an error occurs

C) It is required for all functions

D) It replaces loops

4.4 Concurrency: Threading and Multiprocessing

Threading runs multiple tasks that mostly wait (like network requests) concurrently. Multiprocessing runs tasks that need real CPU power in parallel, using separate processes to bypass Python's Global Interpreter Lock (GIL).

```
import threading
import time

def download(name):
    print(f"Starting {name}")
```

```
    time.sleep(2)
    print(f"Finished {name}")

threads = [threading.Thread(target=download, args=(f"file{i}",)) for i in range(3)]
for t in threads:
    t.start()
for t in threads:
    t.join()     # wait for all threads to finish

# For CPU-heavy work, use multiprocessing instead:
from multiprocessing import Process

def square_numbers():
    for n in range(1000000):
        _ = n * n

if __name__ == "__main__":
    processes = [Process(target=square_numbers) for _ in range(2)]
    for proc in processes:
        proc.start()
    for proc in processes:
        proc.join()
```

Rule of thumb: Use threading (or asyncio) for I/O-bound tasks like network calls and file access. Use multiprocessing for CPU-bound tasks like heavy number crunching.

Q37. When should you prefer multiprocessing over threading in Python?

A) When waiting on network requests

B) When the task is CPU-intensive and needs true parallelism

C) Never - threading is always better

D) Only for reading files

4.5 Working with Web APIs

The 'requests' library (pip install requests) is the standard way to call web APIs and fetch data from the internet.

```
import requests

response = requests.get("https://api.github.com/users/python")
if response.status_code == 200:
    data = response.json()
    print(data["name"], data["public_repos"])
else:
    print(f"Request failed with status {response.status_code}")

# Sending data with POST
payload = {"title": "New Task", "completed": False}
response = requests.post("https://example.com/api/tasks", json=payload)
```

Q38. Which HTTP status code generally indicates a successful request?

A) 404

B) 500

C) 200

D) 301

Practical Task 18: Weather API Client

Fetch and display live data from a public API.

- Install the requests library.
- Pick any free public API (e.g. a joke API or a country-info API) and send a GET request to it.
- Parse the JSON response and print a few chosen fields nicely.
- Wrap the request in a try/except to handle connection errors gracefully.

4.6 Working with Databases (SQLite)

sqlite3 is built into Python and needs no separate installation or server - perfect for learning SQL and small applications.

```
import sqlite3

connection = sqlite3.connect("school.db")
cursor = connection.cursor()

cursor.execute("""
    CREATE TABLE IF NOT EXISTS students (
        id INTEGER PRIMARY KEY,
        name TEXT NOT NULL,
        grade TEXT
    )
""")

cursor.execute("INSERT INTO students (name, grade) VALUES (?, ?)", ("Byron", "A"))
connection.commit()

cursor.execute("SELECT * FROM students")
for row in cursor.fetchall():
    print(row)

connection.close()
```

> **Security note:** Always use parameterised queries (the ? placeholders above) instead of building SQL strings with f-strings or concatenation - this prevents SQL injection attacks.

Q39. Why should you use '?' placeholders in a SQL query instead of an f-string with the values inserted directly?

A) Placeholders run faster on every database

B) It prevents SQL injection attacks

C) f-strings do not work with sqlite3

D) There is no real difference

4.7 Automated Testing

Professional developers write automated tests to prove their code works and to catch regressions when it changes. The two most common tools are the built-in unittest module and the popular third-party pytest.

Using pytest (pip install pytest)

```
# calculator.py
def add(a, b):
    return a + b

# test_calculator.py
```

```
from calculator import add

def test_add_positive_numbers():
    assert add(2, 3) == 5

def test_add_negative_numbers():
    assert add(-1, -1) == -2

# Run from the terminal:
# pytest
```

Using the built-in unittest module

```
import unittest
from calculator import add

class TestCalculator(unittest.TestCase):
    def test_add(self):
        self.assertEqual(add(2, 3), 5)

if __name__ == "__main__":
    unittest.main()
```

Q40. What is the main purpose of automated tests?

A) To make the program run faster

B) To automatically prove your code behaves correctly and catch future regressions

C) To replace the need for documentation

D) To compile Python code

Practical Task 19: Test-Driven Mini Project

Write tests for a small module.

- Write a module string_tools.py with functions is_palindrome() and reverse_words().
- Install pytest and write at least 4 test functions covering normal cases and edge cases (empty string, single word).
- Run pytest from the terminal and confirm all tests pass.
- Intentionally break one function and confirm pytest reports the failure clearly.

4.8 Type Hints

Type hints document what types a function expects and returns. They don't change runtime behaviour but let tools like Pylance catch mistakes before you run the code.

```
def add(a: int, b: int) -> int:
    return a + b

def greet(name: str, times: int = 1) -> str:
    return (name + " ") * times

from typing import List, Dict, Optional

def average(numbers: List[float]) -> float:
    return sum(numbers) / len(numbers)

def find_user(user_id: int) -> Optional[Dict[str, str]]:
```

```
    # Optional[...] means the return value could be None
    return None
```

Q41. Do type hints in Python enforce types at runtime by default?

A) Yes, Python will raise an error if the wrong type is passed

B) No, they are optional documentation checked by external tools like Pylance/mypy, not enforced automatically

C) Only in Python 2

D) Only inside classes

4.8b The logging Module

print() is fine for quick checks, but real applications use the built-in logging module, which adds severity levels, timestamps, and the ability to write to files instead of just the console.

```
import logging

logging.basicConfig(
    filename="app.log",
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s",
)

logging.debug("Detailed diagnostic info")     # not shown - below INFO level
logging.info("Application started")
logging.warning("Disk space running low")
logging.error("Failed to connect to database")
logging.critical("System is shutting down")
```

Logging levels, from least to most severe

Level	Used for
DEBUG	Detailed diagnostic information, useful only while developing
INFO	Confirmation that things are working as expected
WARNING	Something unexpected happened, but the program can continue
ERROR	A serious problem - the program could not perform a function
CRITICAL	A very serious error - the program itself may be unable to continue

Q42. Why is the logging module generally preferred over print() in real applications?

A) print() is deprecated in Python 3

B) logging supports severity levels, timestamps, and writing to files, all configurable without changing code

C) logging runs faster than print in every case

D) print() cannot output numbers

4.9 Asynchronous Programming (asyncio)

async/await is Python's modern way of handling many I/O-bound tasks concurrently on a single thread, widely used in web servers and network clients (e.g. FastAPI, aiohttp).

```
import asyncio
```

```
async def fetch_data(name, delay):
    print(f"Starting {name}")
    await asyncio.sleep(delay)     # non-blocking wait
    print(f"Finished {name}")
    return f"{name} result"

async def main():
    results = await asyncio.gather(
        fetch_data("Task A", 2),
        fetch_data("Task B", 1),
        fetch_data("Task C", 3),
    )
    print(results)

asyncio.run(main())
# Task B finishes first even though it started second,
# because all three run concurrently while waiting.
```

async vs threading: asyncio uses a single thread and cooperative multitasking (tasks voluntarily pause at 'await'), which avoids many of the complications of shared-memory threading, but only helps with I/O-bound work, not CPU-bound work.

Q43. What does the 'await' keyword do inside an async function?

A) Stops the whole program

B) Pauses that task and lets other tasks run until the awaited operation completes

C) Runs the function twice

D) Converts the function to run on a new thread

4.10 Packaging and Distributing Your Project

Once a project grows beyond a single script, structure and packaging matter. A modern Python project typically uses a pyproject.toml file to declare its metadata and dependencies.

```
my_project/
├── pyproject.toml
├── README.md
├── src/
│   └── my_project/
│       ├── __init__.py
│       └── main.py
└── tests/
    └── test_main.py

# pyproject.toml
[project]
name = "my_project"
version = "0.1.0"
description = "A short description of what this project does"
dependencies = [
    "requests>=2.31.0",
]

[build-system]
requires = ["setuptools>=68.0"]
build-backend = "setuptools.build_meta"
```

Once configured, the project can be installed locally in 'editable' mode for development, or built and uploaded to PyPI so others can pip install it:

```
pip install -e .
python -m build
twine upload dist/*
```

4.11 Common Design Patterns in Python

Design patterns are proven, reusable solutions to common software design problems. Three of the most useful for a growing developer are:

Singleton - ensure only one instance of a class ever exists

```
class Config:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance.settings = {}
        return cls._instance

a = Config()
b = Config()
print(a is b)   # True - both variables point to the same object
```

Factory - centralise object creation logic

```
class Dog:
    def speak(self):
        return "Woof"

class Cat:
    def speak(self):
        return "Meow"

def animal_factory(kind):
    animals = {"dog": Dog, "cat": Cat}
    return animals[kind]()

pet = animal_factory("cat")
print(pet.speak())   # Meow
```

Observer - notify multiple objects when something changes

```
class Publisher:
    def __init__(self):
        self.subscribers = []

    def subscribe(self, callback):
        self.subscribers.append(callback)

    def notify(self, message):
        for callback in self.subscribers:
            callback(message)

def email_alert(message):
    print(f"Emailing: {message}")
```

```
publisher = Publisher()
publisher.subscribe(email_alert)
publisher.notify("New order received!")
```

Q44. Which design pattern guarantees that a class only ever has one instance?

A) Factory

B) Observer

C) Singleton

D) Iterator

Practical Task 20: Capstone Challenge

Combine decorators, OOP, file storage, and error handling into one program.

- Build an Inventory class that stores products (name, price, quantity) using a dictionary internally.
- Add methods to add stock, sell stock (raising a custom OutOfStockError if insufficient), and calculate total inventory value.
- Use a @timer decorator (from section 4.1) on the method that calculates total value.
- Persist the inventory to a JSON file after every change, and load it on startup.
- Write at least 3 pytest tests covering normal use and the OutOfStockError case.

PART 5: POPULAR PYTHON LIBRARIES

One reason Python is so widely used is its enormous ecosystem of third-party libraries. This part gives you a practical first taste of four of the most important ones: NumPy and pandas for data work, Matplotlib for visualisation, and Flask for building web applications. Install each with pip before trying the examples.

5.1 NumPy - Numerical Computing

```
pip install numpy
```

NumPy provides the 'array', a fast, memory-efficient alternative to Python lists for numerical work, along with vectorised operations that avoid slow manual loops.

```
import numpy as np

prices = np.array([19.99, 5.50, 12.00, 8.75])
print(prices * 1.15)          # apply 15% VAT to every price at once
print(prices.mean())          # average
print(prices.max(), prices.min())

matrix = np.array([[1, 2], [3, 4]])
print(matrix.T)               # transpose
print(matrix.sum(axis=0))     # sum down each column
```

Q45. What is the main advantage of a NumPy array over a regular Python list for numerical work?

A) It can store text more easily

B) It supports fast, vectorised mathematical operations across all elements at once

C) It is required to print numbers

D) It automatically connects to the internet

5.2 pandas - Working with Tabular Data

```
pip install pandas
```

pandas introduces the DataFrame, a table-like structure similar to a spreadsheet, ideal for loading, cleaning, and analysing CSV or Excel data.

```
import pandas as pd

data = {
    "Name": ["Byron", "Alex", "Sam"],
    "Score": [88, 72, 95],
}
df = pd.DataFrame(data)
print(df)
print(df["Score"].mean())          # average score
print(df[df["Score"] > 80])        # filter rows

# Reading real files:
# df = pd.read_csv("students.csv")
# df.to_csv("output.csv", index=False)
```

Q46. What pandas structure is best described as a spreadsheet-like table with rows and named columns?

A) Series

B) Array

C) DataFrame

D) Dictionary

5.3 Matplotlib - Charts and Graphs

```
pip install matplotlib

import matplotlib.pyplot as plt

months = ["Jan", "Feb", "Mar", "Apr"]
sales = [1200, 1500, 900, 1700]

plt.plot(months, sales, marker="o")
plt.title("Monthly Sales")
plt.xlabel("Month")
plt.ylabel("Sales (R)")
plt.savefig("sales_chart.png")   # or plt.show() in a script with a display
plt.close()
```

Practical Task 21: Mini Data Analysis

Combine pandas and Matplotlib on a small dataset.

- Create a DataFrame of at least 5 students with name and three test scores each.
- Add a column that calculates each student's average score.
- Use Matplotlib to plot a bar chart of student names against their averages, and save it as a .png file.

5.4 Flask - Building a Web Application

```
pip install flask
```

Flask is a lightweight 'micro' web framework, ideal for learning how web servers work and for building small to medium web applications and APIs.

```
# app.py
from flask import Flask, jsonify, request

app = Flask(__name__)

tasks = []

@app.route("/")
def home():
    return "Welcome to my Flask API!"

@app.route("/tasks", methods=["GET"])
def get_tasks():
    return jsonify(tasks)

@app.route("/tasks", methods=["POST"])
def add_task():
    new_task = request.json
    tasks.append(new_task)
    return jsonify(new_task), 201

if __name__ == "__main__":
    app.run(debug=True)
```

```
# Run with: python app.py
# Then visit http://127.0.0.1:5000/ in a browser
```

Q47. In a Flask app, what does the @app.route("/tasks") decorator do?

A) Deletes the /tasks page

B) Maps the /tasks URL to the function defined immediately below it

C) Installs Flask automatically

D) Runs the server on a different port

Practical Task 22: Mini REST API

Build a simple task API with Flask.

- Set up a Flask app with GET /tasks (list all tasks) and POST /tasks (add a task) routes.
- Add a DELETE /tasks/<int:index> route that removes a task by its position in the list.
- Test each route using a tool like Postman, curl, or your browser (for GET requests).

5.5 openpyxl - Automating Excel Spreadsheets

```
pip install openpyxl
```

openpyxl lets you create, read, and update real .xlsx Excel files programmatically - useful for automating repetitive reporting tasks.

```
from openpyxl import Workbook, load_workbook

# Creating a new spreadsheet
workbook = Workbook()
sheet = workbook.active
sheet.title = "Scores"
sheet.append(["Name", "Score"])
sheet.append(["Byron", 88])
sheet.append(["Alex", 72])
workbook.save("scores.xlsx")

# Reading an existing spreadsheet
workbook = load_workbook("scores.xlsx")
sheet = workbook.active
for row in sheet.iter_rows(min_row=2, values_only=True):
    name, score = row
    print(f"{name}: {score}")
```

Q48. Which library would you reach for to automatically generate a real .xlsx Excel report from Python?

A) matplotlib

B) openpyxl

C) flask

D) re

Practical Task 23: Automated Report Generator

Combine several libraries into one small automation tool.

- Use pandas to build a small dataset of monthly expenses (category, amount).
- Export it to an Excel file using openpyxl or pandas' built-in to_excel() method.

- Use Matplotlib to also save a pie chart image summarising spending by category.

PART 6: PRACTICAL PROJECTS

Reading and small exercises build understanding; full projects build confidence. Each project below combines multiple concepts from this guide, including the libraries covered in Part 5. Build them in order - each one is slightly harder than the last.

Project 1: Command-Line To-Do List

Level: Beginner **Skills used:** Lists, dictionaries, loops, functions, file I/O (JSON)

Build a to-do list you run from the terminal that saves tasks between sessions.

- Store tasks as a list of dictionaries: {"task": "Buy milk", "done": False}.
- Build a menu loop offering: Add task, View tasks, Mark task complete, Delete task, Quit.
- Save the list to tasks.json every time it changes, and load it on startup.
- Stretch goal: add due dates and sort tasks by date.

Project 2: Library Management System

Level: Beginner-Intermediate **Skills used:** OOP, classes, exception handling, file storage

Model a small library with Book and Member classes.

- Create a Book class (title, author, ISBN, is_borrowed) and a Member class (name, member_id, borrowed_books).
- Add methods to borrow and return books, raising a custom exception if a book is already borrowed.
- Store the full catalogue in a list, and provide a search function using string matching.
- Persist the catalogue to a JSON file between runs.

Project 3: Movie Rental System

Level: Intermediate **Skills used:** OOP, menus, validation, aggregation

Simulate a movie rental store with customers and a rentable catalogue.

- Create a Movie class (title, genre, available_copies) and a Customer class (name, rented_movies, rent(), return_movie()).
- Build a menu-driven main program: list movies, rent a movie, return a movie, view a customer's history.
- Validate that a movie cannot be rented if available_copies is 0.
- Add a late fee calculator based on days overdue.

Project 4: Weather Dashboard (API project)

Level: Intermediate-Advanced **Skills used:** requests, JSON parsing, error handling, functions

Build a small tool that fetches and displays real weather data for any city.

- Sign up for a free API key from a weather API provider.
- Write a function get_weather(city) that calls the API and returns a dictionary of the key details.
- Handle network errors and invalid city names gracefully with try/except.
- Loop so the user can check multiple cities without restarting the program.

Project 5: Notes App with a Database

Level: Advanced **Skills used:** sqlite3, CRUD operations, OOP

Build a persistent notes app backed by a real (embedded) database instead of a flat file.

- Design a notes table (id, title, content, created_at) in SQLite.
- Write functions for Create, Read, Update, and Delete (CRUD) operations using parameterised queries.
- Wrap the database logic in a NotesRepository class to keep it separate from the menu code.
- Write pytest tests for each CRUD function using a temporary test database.

APPENDIX A: PYTHON QUICK-REFERENCE CHEAT SHEET

Common Errors and What They Mean

Error	Typical cause	Fix
SyntaxError	Missing colon, bracket, or quote	Re-read the flagged line and the one before it
IndentationError	Mixed tabs/spaces or inconsistent indent	Use 4 spaces consistently (VS Code does this automatically)
NameError	Using a variable before defining it, or a typo	Check spelling and definition order
TypeError	Mixing incompatible types (e.g. str + int)	Convert types explicitly with str(), int(), float()
ModuleNotFoundError	Package not installed, or venv not activated	pip install the package inside the active virtual environment
IndexError	Accessing a list/tuple index that doesn't exist	Check len() before indexing, or use try/except

Handy Built-in Functions

Function	Purpose
len()	Number of items in a sequence
type()	The data type of a value
range()	A sequence of numbers
sorted()	Returns a new sorted list
enumerate()	Loop with an automatic index
zip()	Loop over two lists in parallel
sum() / min() / max()	Aggregate values in a sequence
isinstance()	Check if a value is a given type

PEP 8 Style Essentials

- Use 4 spaces per indentation level, never tabs.
- Use snake_case for variables and functions, PascalCase for classes.
- Keep lines under 79-99 characters where practical.
- Use two blank lines between top-level function/class definitions.
- Write docstrings for every public function, class, and module.
- Let a formatter (Black) and linter (Ruff) enforce this automatically.

Glossary of Key Terms

Term	Meaning
Interpreter	The program that reads and executes your Python code line by line
Variable	A named container that stores a value
Function	A reusable, named block of code that performs a task

Class	A blueprint for creating objects that bundle data and behaviour
Object / Instance	A specific thing created from a class
Module	A single .py file containing reusable code
Package	A folder of related modules
Library / Framework	A collection of pre-written code you can install and use (e.g. requests, Django)
Exception	An error that occurs while a program is running
Iterable	Anything that can be looped over (lists, strings, dicts, files)
Mutable / Immutable	Whether a value can be changed after creation (lists = mutable, tuples = immutable)
Virtual environment	An isolated Python setup for a single project's dependencies
IDE / Editor	Software used to write and run code, such as VS Code
API	Application Programming Interface - a way for programs to talk to each other, often over the web
Refactoring	Rewriting code to improve its structure without changing what it does

Frequently Asked Interview-Style Questions

Q: What is the difference between a list and a tuple?

A: Lists are mutable (changeable) and typically used for collections that grow or shrink; tuples are immutable and typically used for fixed groups of values, such as coordinates.

Q: What is PEP 8?

A: PEP 8 is Python's official style guide, covering naming conventions, indentation, and formatting to keep code consistent and readable across the community.

Q: What is the difference between == and is?

A: == compares values for equality; 'is' checks whether two variables point to the exact same object in memory.

Q: What is a decorator used for?

A: A decorator wraps a function to add extra behaviour (logging, timing, access control) without modifying the function's own code.

Q: What is the GIL?

A: The Global Interpreter Lock is a mechanism in CPython that allows only one thread to execute Python bytecode at a time, which is why CPU-bound work benefits more from multiprocessing than threading.

Q: What is a virtual environment and why use one?

A: It is an isolated copy of Python and its packages for one project, preventing dependency conflicts between different projects on the same machine.

Q: What is the difference between a shallow copy and a deep copy?

A: A shallow copy duplicates the outer object but still references the same nested objects; a deep copy recursively duplicates every nested object too, so the two copies are fully independent.

Q: What does *args and **kwargs mean?

A: *args collects extra positional arguments into a tuple; **kwargs collects extra keyword arguments into a dictionary, letting a function accept a flexible number of inputs.

Appendix B: Coding Challenge Practice

These short challenges are the kind commonly used to test fundamentals in technical interviews and coding assessments. Try to solve each one yourself before reading the solution.

Challenge 1: FizzBuzz

Print the numbers from 1 to 30. For multiples of 3, print 'Fizz' instead of the number. For multiples of 5, print 'Buzz'. For multiples of both, print 'FizzBuzz'.

```
for i in range(1, 31):
    if i % 15 == 0:
        print("FizzBuzz")
    elif i % 3 == 0:
        print("Fizz")
    elif i % 5 == 0:
        print("Buzz")
    else:
        print(i)
```

Challenge 2: Reverse a String Without Slicing

Write a function that reverses a string without using slicing ([::-1]).

```
def reverse_string(text):
    result = ""
    for char in text:
        result = char + result
    return result

print(reverse_string("hello"))   # olleh
```

Challenge 3: Find the Most Frequent Item

Given a list of items, return the item that appears most often.

```
from collections import Counter

def most_frequent(items):
    counts = Counter(items)
    return counts.most_common(1)[0][0]

print(most_frequent(["a", "b", "a", "c", "a", "b"]))   # a
```

Challenge 4: Check for Balanced Brackets

Write a function that checks whether a string of brackets ({ [is balanced (every opening bracket has a matching closing bracket in the right order).

```
def is_balanced(text):
    stack = []
    pairs = {")": "(", "]": "[", "}": "{"}
    for char in text:
        if char in "([{":
            stack.append(char)
        elif char in ")]}":
            if not stack or stack.pop() != pairs[char]:
```

```
                return False
    return not stack

print(is_balanced("([{}])"))   # True
print(is_balanced("([)]"))     # False
```

Challenge 5: Two Sum

Given a list of numbers and a target value, return the indices of the two numbers that add up to the target.

```
def two_sum(numbers, target):
    seen = {}
    for index, value in enumerate(numbers):
        complement = target - value
        if complement in seen:
            return [seen[complement], index]
        seen[value] = index
    return None

print(two_sum([2, 7, 11, 15], 9))   # [0, 1]  because 2 + 7 = 9
```

Where to Go Next

- Build the five practical projects in Part 5 end-to-end, without copying code.
- Read other people's open-source Python code on GitHub to see real style and structure.
- Pick a specialisation: web development (Flask/Django), data science (pandas/NumPy), automation/scripting, or general software engineering.
- Contribute a small fix to an open-source project once you feel confident with Git.

ANSWER KEY

Answers to every multiple-choice question in this guide, grouped by chapter, in the order they appeared.

Setup

Q1: B) Git is the version-control tool that runs locally; GitHub is a website that hosts Git repositories online - Git is the underlying version-control system installed on your machine; GitHub (and alternatives like GitLab) provide online hosting, collaboration, and backup for Git repositories.

Q2: C) git commit -m "message" - git commit records the staged changes as a new point in the project's history, along with a descriptive message.

Q3: B) Add python.exe to PATH - Adding Python to PATH lets the operating system find the python.exe command from any terminal location.

Q4: B) macOS reserves 'python' for its outdated system copy - macOS ships an old legacy Python under the plain 'python' name, so the fresh install is accessed via python3.

Q5: B) To isolate a project's packages from other projects - Virtual environments give each project its own independent set of installed packages, avoiding version conflicts.

Q6: B) pip install -r requirements.txt - The -r flag tells pip to read package names from the given requirements file.

Fundamentals

Q7: C) bool - True and False are the two values of the bool (Boolean) type.

Q8: C) _total - Names may start with an underscore or letter, not a digit; hyphens aren't allowed; 'class' is a reserved keyword.

Q9: B) 4 - Floor division discards the remainder and any decimal part, so 17 // 4 = 4.

Q10: C) 1 - The modulus operator returns the remainder of the division: 17 / 4 = 4 remainder 1.

Q11: C) ell - Slicing [1:4] takes index 1 up to (not including) index 4: characters at positions 1, 2, 3 -> 'ell'.

Q12: B) Indentation - Python uses whitespace indentation to group statements into blocks, unlike many other languages that use braces.

Q13: B) 0,1,2,3,4 - range(5) generates numbers starting at 0 up to, but not including, 5.

Q14: B) Skips the rest of the current iteration and moves to the next one - 'continue' jumps immediately to the next iteration, skipping any remaining code in the current one.

Q15: B) return - return exits the function and passes a value back to the caller. print only displays text and does not hand back a usable value.

Q16: B) Any number of positional arguments - *args collects any number of extra positional arguments into a tuple inside the function.

Q17: C) dict - A dictionary maps keys (names) to values (numbers), which is exactly what a phone book needs.

Q18: B) Sets cannot contain duplicate values by definition - A set automatically enforces uniqueness, so converting a list to a set is a quick way to strip duplicates.

Q19: C) Tuples are immutable (cannot be changed after creation), lists are mutable - Tuples cannot be modified after creation (immutable); lists can have items added, removed, or changed (mutable).

Q20: C) Raise a ValueError - int() cannot parse a string containing a decimal point directly; you would need float("3.5") first, then int() on the result.

Q21: D) SQL - It drills into the first employee's skills list and takes the item at index 1, which is 'SQL'.

Intermediate

Q22: D) finally - finally always executes, making it ideal for cleanup actions like closing files or connections.

Q23: B) KeyError - KeyError is specifically raised for missing dictionary keys.

Q24: C) "a" - 'a' (append) mode adds to the end of the file; 'w' (write) would overwrite the file's existing content.

Q25: B) The first requires math.sqrt(), the second lets you call sqrt() directly - 'import math' brings in the whole module, so its functions need the math. prefix. 'from math import sqrt' brings just that one function into your local namespace.

Q26: B) Runs automatically when a new object is created, to set up its initial attributes - __init__ is the constructor - Python calls it automatically whenever a new instance of the class is created.

Q27: B) Polymorphism - Polymorphism allows different classes to implement the same method name in their own specific way.

Q28: B) super() - super() gives access to the parent class's methods, commonly used to call the parent's __init__.

Q29: B) [0, 1, 4, 9, 16] - This list comprehension squares every number from 0 to 4, producing [0, 1, 4, 9, 16].

Q30: C) re.findall() - re.findall() scans the whole string and returns a list of all non-overlapping matches.

Q31: B) strftime() - strftime() ('string format time') turns a datetime object into a string using a format pattern like %Y-%m-%d.

Q32: C) StopIteration - StopIteration signals to the for loop (or manual next() calls) that the iterator is exhausted.

Q33: B) It keeps secrets out of source control, reducing the risk of them leaking publicly - Keeping secrets outside the codebase means they aren't accidentally committed to Git/GitHub or shared when the code is shared.

Advanced

Q34: B) Wraps a function to add extra behaviour without modifying its code - Decorators wrap the original function, letting you add behaviour (logging, timing, access checks) before or after it runs.

Q35: B) yield - yield pauses the function and returns a value, resuming from that point the next time a value is requested.

Q36: B) It guarantees setup and cleanup code runs, even if an error occurs - Context managers guarantee cleanup (like closing a file or connection) happens automatically, even if an exception is raised inside the block.

Q37: B) When the task is CPU-intensive and needs true parallelism - Python's GIL prevents true parallel execution of Python bytecode across threads, so CPU-bound work benefits from separate processes instead.

Q38: C) 200 - 200 OK indicates the request succeeded. 404 means Not Found, and 500 means Internal Server Error.

Q39: B) It prevents SQL injection attacks - Parameterised queries safely escape user input, preventing malicious data from being interpreted as SQL commands.

Q40: B) To automatically prove your code behaves correctly and catch future regressions - Tests verify correctness automatically and re-run quickly whenever code changes, catching bugs early.

Q41: **B) No, they are optional documentation checked by external tools like Pylance/mypy, not enforced automatically** - Python remains dynamically typed at runtime; type hints are advisory and checked by separate static analysis tools.

Q42: **B) logging supports severity levels, timestamps, and writing to files, all configurable without changing code** - logging gives structured, configurable output (levels, timestamps, destinations) suited to real applications, rather than plain, always-on console text.

Q43: **B) Pauses that task and lets other tasks run until the awaited operation completes** - await hands control back to the event loop, allowing other async tasks to make progress while the current one waits.

Q44: **C) Singleton** - The Singleton pattern restricts instantiation so every call to create the object returns the same shared instance.

Libraries

Q45: **B) It supports fast, vectorised mathematical operations across all elements at once** - NumPy arrays let you apply an operation to every element simultaneously (vectorisation), which is far faster than looping in pure Python.

Q46: **C) DataFrame** - A DataFrame is pandas' two-dimensional, labelled table structure - its core data type.

Q47: **B) Maps the /tasks URL to the function defined immediately below it** - @app.route() registers a URL pattern, connecting incoming web requests for that path to the decorated view function.

Q48: **B) openpyxl** - openpyxl is purpose-built for creating and editing native Excel (.xlsx) files.

Closing Notes

You've now covered the full journey from your first print() statement to decorators, async programming, databases, testing, and real libraries like pandas and Flask. That is a genuinely complete foundation - the same one professional Python developers build on every day.

Learning to program is not linear. Re-read chapters that didn't fully click the first time, redo practical tasks from memory a week later, and don't skip the projects in Part 6 - they are where isolated facts turn into real skill.

Keep this guide as a reference: come back to the cheat sheet in Appendix A whenever you hit a familiar error, and revisit the coding challenges in Appendix B before interviews or assessments.

Good luck, and happy coding.

- End of Guide -